

OS and CPU Recap

Устройство CPU

CPU:

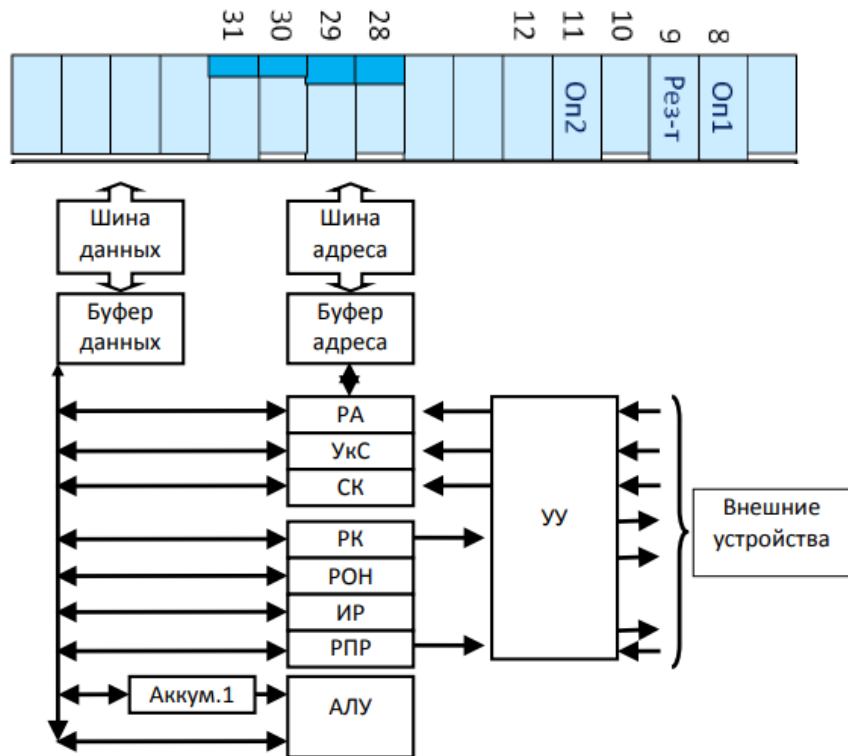
- АЛУ (ALU) – арифметико-логическое устройство
- Регистры — внутренняя память процессора – n-разрядные ячейки, где хранятся операнды и промежуточные значения
- УУ (CU) – устройство управления

Тракт данных (data path):

- читает/пишет в память
- выполнение арифметических и логических операций в АЛУ
- хранение промежуточных результатов в регистрах

Управление данных (УУ, CU):

- управляет регистрами, трактом данных, АЛУ
- берет байты инструкций и данных из памяти
- превращает операции (опкоды) в управляющие сигналы



Пример

abstract code

$x = 7$

$y = 5$

$z = x + y$



assembler code

x: dq 7

y: dq 5

z: dq 0

mov rax, [x] ; RAX <- *(addr x) (load)

mov rbx, [y] ; RBX <- *(addr y) (load)

add rax, rbx ; RAX <- RAX + RBX (ALU, updates flags)

mov [z], rax ; *(addr z) <- RAX (store)

Устройство памяти

Логически:

Линейный массив ячеек, где каждая ячейка имеет свой адрес

Физически:

Двумерный массив ячеек, каждая из которых хранит 0 или 1

Примеры

DRAM (Dynamic Random Access Memory) – конденсатор + транзистор – медленная, дешевая

SRAM (Static Random Access Memory) – набор триггеров (кеш-память, регистры) – быстрая, но дорогая

Линейная организация памяти

0000	—	0	1	1	0	1	0	0	0
0001	—	1	0	0	0	1	1	1	0
0002	—	1	1	0	1	0	0	0	1
0003	—	0	0	1	0	1	1	1	1
0004	—	1	1	1	1	1	0	0	0

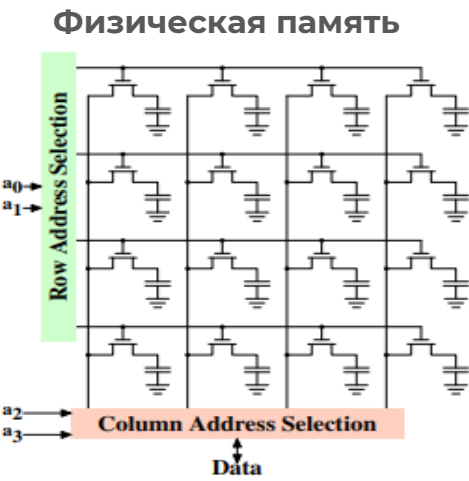


Figure 2.7: Dynamic RAM Schematic

ОС

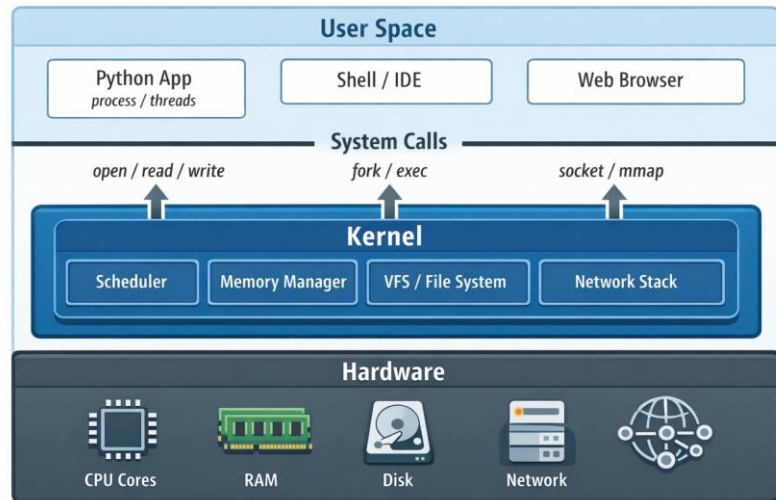
ОС (операционная система) — системное ПО, которое:

- управляет ресурсами компьютера (CPU, память, устройства ввода/вывода)
- предоставляет абстракции: *процессы, потоки, виртуальная память, файлы, сокет*

Обычно выделяют два режима выполнения:

user space и **kernel space**

- **User space:** выполняются приложения (пользовательские процессы: shell, IDE, Python и т.д.)
- **Kernel space:** ядро – наиболее привилегированная часть ОС, **организует выполнение** программ и управляет доступом к ресурсам

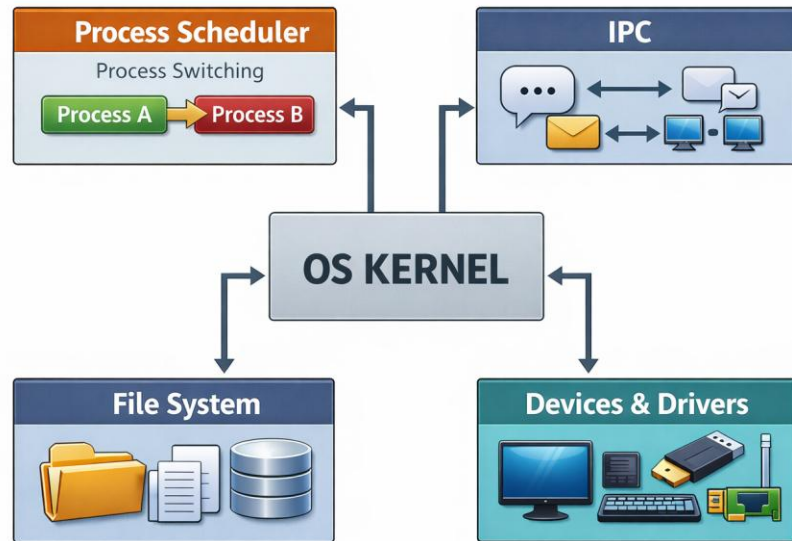


Взаимодействие с ядром происходит через **system calls (системные вызовы)**, например **open()**, **fork()**

Ядро ОС

Ядро ОС предоставляет:

- **Планировщик процессов (*Process Scheduler*)**
- **Межпроцессное взаимодействие (*IPC* — *Inter-Process Communication*)**
- Файловая система (File System)
- Менеджер памяти (Memory Manager)
- Блок работы с устройствами (I/O Interface, Network Interface и Drivers)



Процесс

Процесс – это выполняющийся экземпляр программы со своим адресным пространством и ресурсами, создаваемые и управляемый ОС

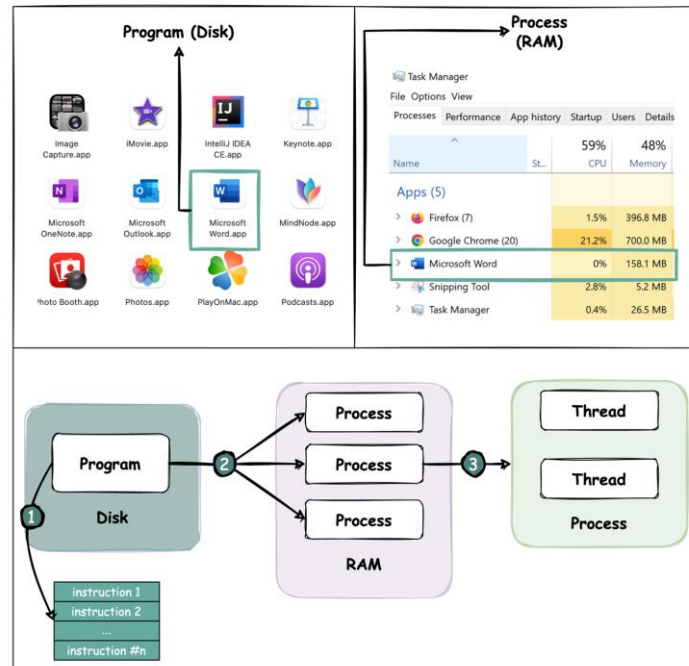
Программа – описание последовательности инструкций и данных в виде кода, которые обрабатывает CPU

ОС хранит для процесса **process context**:

- **виртуальное адресное пространство:**
code/data/heap/stack
- PID (Process ID), PPID (Parent Process ID)
- ресурсы ОС: открытые файлы/сокеты
- состояние процесса – Running/ Sleeping и тд
- права и окружение: UID/GID, env
- как минимум один **поток**
 - у потока: регистры CPU + стек (контекст выполнения)

Program vs Process vs Thread

 blog.bytebytego.com



Поток

Поток — это объект, создаваемый и планируемый ОС; минимальная единица выполнения инструкций на CPU. Поток принадлежит процессу

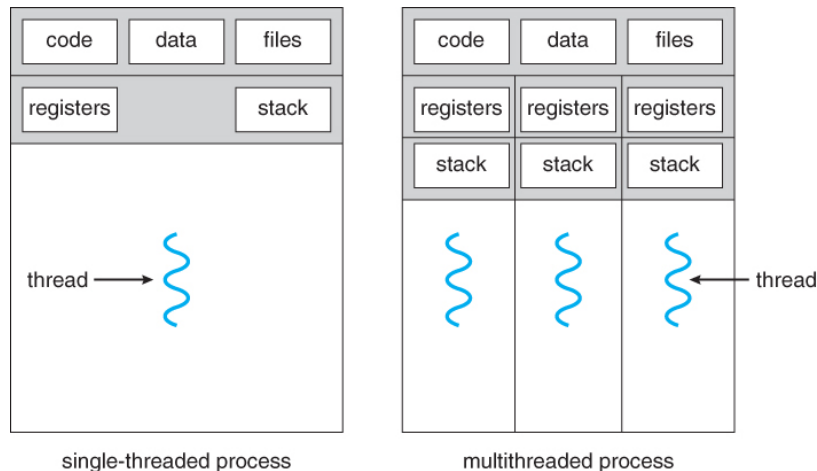
В то время как **процесс** — это **контейнер** ресурсов, но сами инструкции выполняет именно **поток**

ОС хранит для потока **thread context**:

- набор регистров, в том числе регистр IP (IP/PC – instruction pointer, SP – stack pointer)
- TID (Thread ID)
- stack
- состояние (Running / Sleeping)

Свойства потоков:

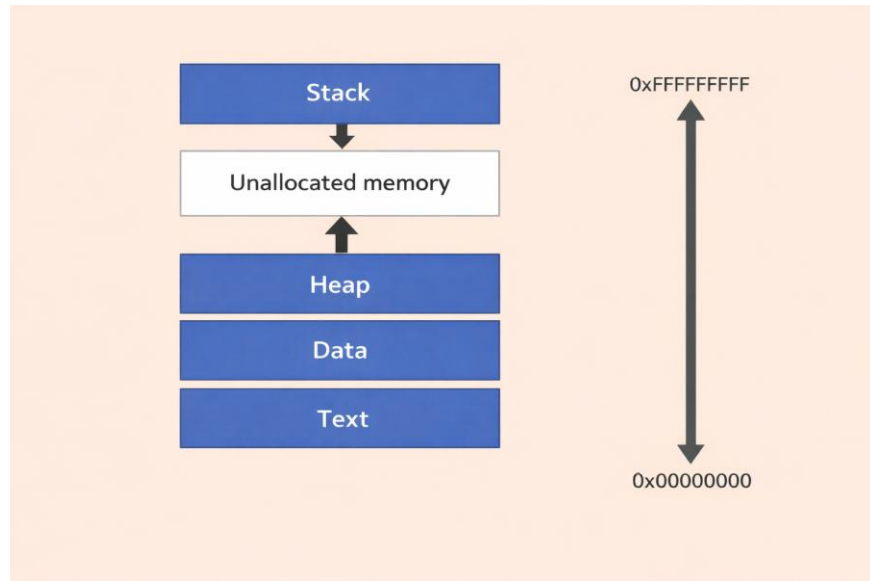
- могут выполнять несколько задач внутри одного процесса – **concurrency** (**конкурентность**)
- разделяют память без IPC
- быстрее переключение, чем между процессами



Память процесса с точки зрения ОС. Виртуальное адресное пространство

Логически память разбивается на следующие сегменты:

- **Stack (Стек)** – отвечает за выполнение процесса (стек фрейм), поэтому у каждого потока внутри процесса свой стек
- **Heap (Куча)** – динамическое размещение данных – управляет аллокатор (**malloc** в C)
- **Data (Данные)** – глобальные/статические переменные с заданными начальными значениями (явным образом не управляем, если пишем на python)
- **Text/Code (Код)** – машинный код (машинный код интерпретатора CPython)



Блок контроля процессов (PCB)

При создании процесса ОС создает **PCB (*Process Control Block*)**, который описывает процесс следующей структурой:



Состояние процесса/потока

R (Running / Runnable) — процесс выполняется на CPU или готов выполняться (ждёт своей очереди на CPU)

S (Sleeping, interruptible) — спит, ждёт событие (например, I/O). *Можно разбудить сигналом (например, SIGKILL)*

D (Sleeping, uninterruptible) — спит “жёстко”, обычно ждёт I/O в ядре. *Сигналами почти не будится, пока не закончится ожидание*

T (Stopped) — остановлен (например, Ctrl+Z)

Z (Zombie) — процесс уже завершился, но родитель ещё не сделал **wait()**, поэтому запись о нём остаётся в таблице процессов (почти не ест CPU/память, но занимает слот записи)

[Main]		[I/O]												
PID	USER	PRI	NI	VIRT	RES	SHR	Sz	CPU%	MEM%	TIME+	Command			
604	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
605	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
606	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
607	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
608	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
615	evgeniiswi	20	0	6200	2432	2432	S	0.0	0.0	0:00.04		└ /bin/bash --init-file /home/evgeniiswi/.vscode-		
618	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
680	evgeniiswi	20	0	6200	2560	2560	S	0.0	0.0	0:00.02		└ /bin/bash --init-file /home/evgeniiswi/.vscode-		
681	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		
863	evgeniiswi	20	0	7392	5248	3200	S	0.0	0.1	0:00.49		└ /bin/bash --init-file /home/evgeniiswi/.vscode-		
41906	evgeniiswi	20	0	6124	4736	3456	R	1.3	0.1	0:01.57		└ htop		
32448	evgeniiswi	20	0	6144	4736	3456	T	0.0	0.1	0:00.09		└ htop		
864	evgeniiswi	20	0	1162M	48300	16640	S	0.0	0.6	0:00.00		└ /home/evgeniiswi/.vscode-server/bin/994fd12f8d3		

I/O-bound

I/O (Input/Output) – это ввод/вывод данных, т.е. обмен данным между процессом и объектом (устройством) вне CPU

Типичные I/O:

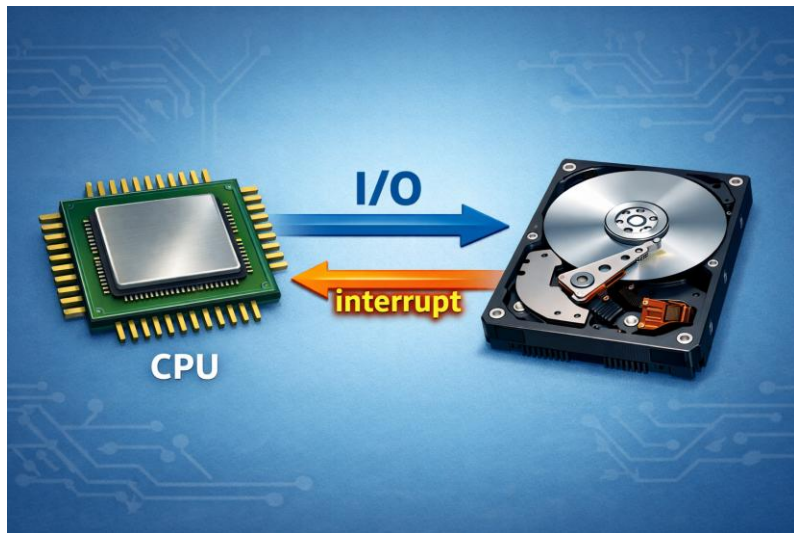
- чтение/запись файла с диска/на диск (`read()`, `write()`)
- сетевой запрос (`send()`, `recv()`)
- ввод с клавиатуры
- вывод звука со звуковой карты

Свойства

- I/O-задача (I/O-bound) – это операция, где процесс ждет внешнюю систему, при этом CPU непосредственно не участвует в этом процессе
- I/O-bound задача происходит дольше на порядок, чем сри операции

Абстрактный пример

Пусть на CPU есть один процесс, в котором может жить несколько потоков. Во время I/O поток **блокируется** (уходит в Waiting (Sleeping) состояние). Если есть еще поток, ядро переключает CPU на другой поток. Если нет, то процесс “засыпает”. Это управляется **планировщиком**



Планировщик ОС

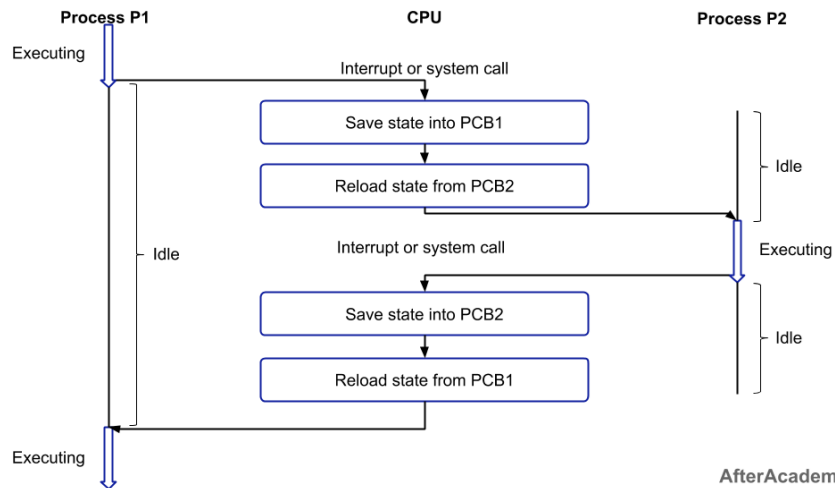
Планировщик — планирует, какой процесс/поток получит CPU, а также реализует **context switch**

Принцип работы

Потоку дается CPU, **таймер** тикает, и раз в тик делает **interrupt**. Управление уходит в **обработчик прерывания** в ядре ОС (**IRQ handler (Interrupt Request)**), таймер обновляется и если есть еще процессы/потоки в очереди, делается **context switch**. В этом случае процесс/поток прервался по **тайм-кванту**

Но:

- Поток может уступить CPU, если делает блокирующий I/O через system call – переходит в *Waiting*
- Когда I/O готово, устройство уведомляет ОС (обычно **hardware interrupt**) – поток “будится” и снова становится *Runnable*



I/O-bound vs CPU-bound

CPU-bound задача

Процесс занят **вычислениями**:

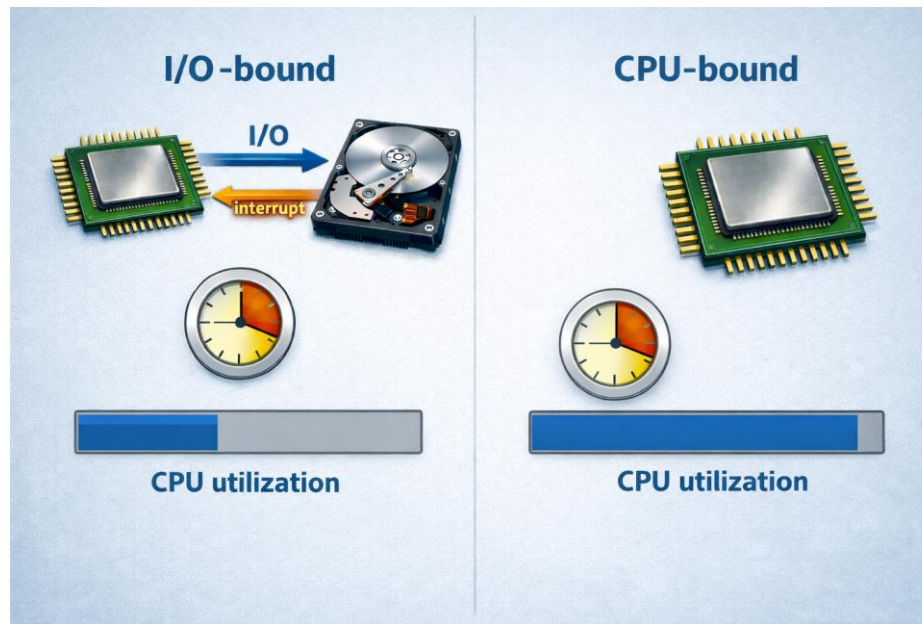
- CPU используется активно
- скорость ограничена производительностью CPU

Примеры: рендеринг, обработка аудио/видео, считаем что-нибудь в цикле на python

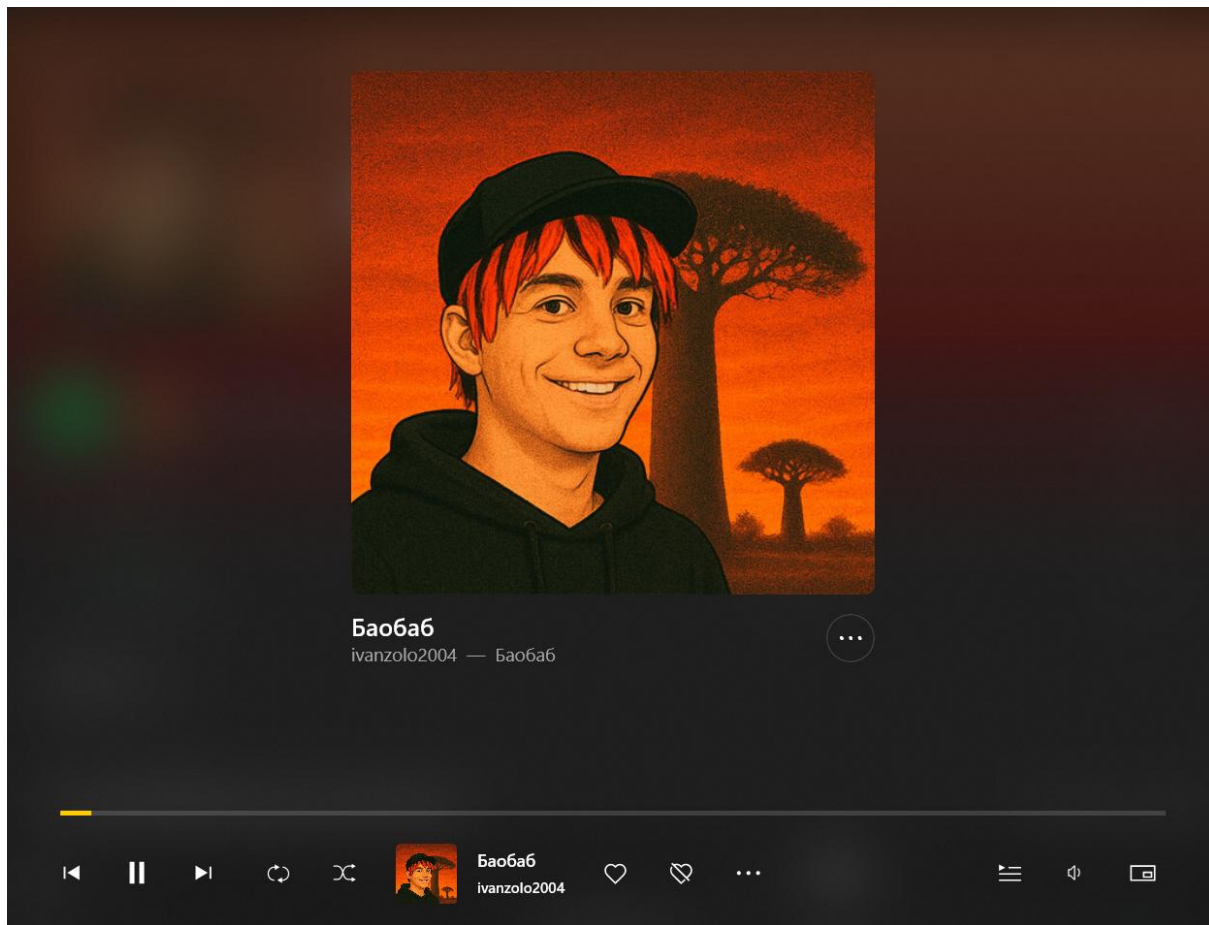
I/O-bound задача

Процесс в основном **ждёт**:

- CPU почти не используется (в ожидании), если нет других потоков
- скорость ограничена I/O

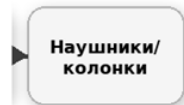


Пример



Пример

Яндекс Музыка → аудиосистема Windows (упрощённая схема)



Пример

Яндекс Музыка → аудиосистема Windows (упрощённая схема)



- Делается запрос в Яндекс музыку API посредством сокетов – **I/O-bound**
- По сети приходит датка трека в виде MP3
- Декодирование и перевод в **PCM (Pulse-Code Modulation)** – последовательность семплов – **CPU-bound**
- ЯМузыка общается с **запущенным процессом audiodg.exe** через WASAPI: через WASAPI ЯМузыка пишет в буфер PCM-фреймы. Эти фреймы считывает аудио-движок и делает всю необходимую обработку (микширование, ресемплинг, эквализация и др. эффекты)
- Поток PCM-фреймов передается в драйвер звуковой карты
- Звуковая карта – вывод звука, перевод из цифры в аналог
- Наушники

Многозадачность: concurrency и parallelism

Concurrency (конкурентность) — несколько задач **в процессе** одновременно

- планировщик постоянно переключается между задачами по тайм-кванту или во время I/O
- возможна даже на одном ядре

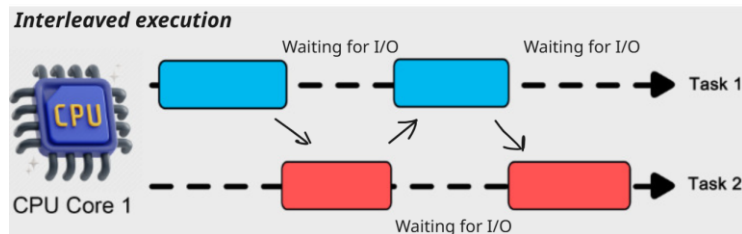
Parallelism (параллелизм) — несколько задач выполняются **физически** одновременно

- требуется несколько ядер CPU (физических или логических, например, **SMT (Simultaneous Multithreading) – Intel Hyper-Threading**)

Хотим эффективно использовать CPU

- пока один поток/процесс ждет I/O, другой может выполняться (**concurrency**)
- если есть несколько ядер → хотим нагрузить их работой (**parallelism**)

Concurrency



Parallelism

